ChatBridge Pre-Search Document

Case Study Analysis

TutorMeAI serves over 200,000 students and teachers across 10,000 school districts, and their next competitive move is to let third-party applications live inside the chat experience. On the surface this sounds like a feature request — embed some apps, let teachers pick which ones to enable, move on. But underneath it sits a set of genuinely hard problems that forced us to make deliberate trade-offs between openness and control, speed and safety, flexibility and predictability.

The central tension is trust. TutorMeAI's users are children. Every technical decision about how third-party code enters the platform carries an ethical dimension that most app-integration projects never have to consider. A malicious or poorly written app could expose student data, display harmful content, or simply break the learning experience in ways a child cannot diagnose. We chose iframe-based sandboxing as our isolation boundary — not because it is the most powerful option, but because it is the most honest one. Iframes enforce a real process-level boundary between the host and the third-party code. The app cannot touch the parent DOM, cannot read chat history, and cannot access other apps' state. We restrict the sandbox to `allow-scripts allow-same-origin allow-popups allow-forms` — enough for rich interactivity, but no access to top-level navigation or arbitrary code execution in the host context. We considered Web Components and server-side rendering as alternatives, but Web Components share the same JavaScript execution context as the host (one prototype pollution away from a full compromise), and server-side rendering introduces latency and cost that scale poorly for interactive apps like chess boards or physics simulators. The trade-off is clear: iframes add complexity to communication, but they provide a security boundary that is structurally enforceable rather than policy-dependent.

The second hard problem is communication. The chatbot must know what tools an app provides, invoke them at the right time, render the app's response, and understand when the app is done — all without prior knowledge of what any given third-party will build. We designed a postMessage-based protocol with four lifecycle events: `ready`, `invoke`, `state`, and `complete`. This creates a clean contract — the platform never reaches into the app, and the app never reaches into the platform. State updates flow through narrow, typed channels. We chose this over WebSocket-based communication because postMessage is synchronous to the browser's event loop, requires no server infrastructure, and works identically in development and production. The cost is that postMessage is origin-scoped and untyped at the transport layer, so we enforce structure through TypeScript interfaces on both sides.

The ethical decision we wrestled with most was how much control to give teachers versus how much to enforce at the platform level. We landed on a two-layer model: the platform enforces structural safety (sandboxing, protocol compliance, timeout enforcement), while teachers control policy (which apps are available, which students can access them). This means a teacher cannot accidentally enable an app that bypasses the sandbox, but they can choose to enable or disable any app that passes the structural checks. We deliberately chose not to implement content filtering at the iframe level — that responsibility belongs to the app developer and the review process, not to runtime interception that would be brittle and easy to circumvent.

Finally, we had to decide how to handle the cost implications of tool-calling at scale. Every plugin adds token overhead to every API call — tool schemas are injected into the system prompt. With three plugins active, that is roughly 260 extra input tokens per request. At 200,000 daily users, this is real money. We chose to make plugin loading lazy and session-scoped: tools are only injected when a user's conversation context suggests they might be needed, and teachers can disable plugins they do not use. This is a pragmatic trade-off between AI capability and cost control that respects TutorMeAI's position as education infrastructure where budgets are tight and waste is unacceptable.

Phase 1: Define Your Constraints

1. Scale and Load Profile

- **Users at launch**: ~500 beta users (single school district pilot); targeting 10,000+ within 6 months as TutorMeAI rolls out to existing districts
- **Traffic pattern**: Spiky — heavy usage during school hours (8 AM–3 PM local time), near-zero overnight. Monday–Friday dominant with exam-period surges
- **Concurrent app sessions per user**: Typically 1 active app at a time, but users may switch between 2–3 apps within a single conversation session
- **Cold start tolerance**: Under 2 seconds for app iframe loading. Students lose focus quickly — anything over 3 seconds and they will assume it is broken

2. Budget and Cost Ceiling

- **Monthly spend limit**: $50/month for development and testing; production costs scale with TutorMeAI's existing per-district pricing model
- **Pay-per-use**: Acceptable — users supply their own API keys (BYOK model), so LLM costs pass through to the district
- **LLM cost per tool invocation**: Under $0.005 per invocation on mid-tier models (GPT-4o-mini, Claude Haiku). Tool schema overhead adds ~~260 tokens per request with all 3 plugins active (~~$0.001 on GPT-4o)
- **Time vs money trade-off**: Forking Chatbox saves weeks of chat infrastructure work; invest time in the plugin system instead of rebuilding what exists

3. Time to Ship

- **MVP timeline**: 24 hours (pre-search + basic architecture)
- **Early submission**: 4 days (full plugin system + 3 working apps)
- **Final**: 7 days (polish, OAuth flows, deployment, documentation)
- **Priority**: Speed-to-market for the sprint, but architecture decisions should not create tech debt that blocks future plugin developers
- **Iteration cadence**: Daily — ship vertical slices (one complete plugin integration per day)

4. Security and Sandboxing

- **Isolation method**: Iframe sandbox with restricted attribute set (`allow-scripts allow-same-origin allow-popups allow-forms`). No `allow-top-navigation` , no `allow-modals` for untrusted content
- **Malicious app registration**: Structural enforcement — plugins must conform to the typed `PluginDefinition` interface and pass schema validation. Built-in plugins are bundled; third-party plugins would go through a review/approval process before being added to the registry
- **CSP requirements**: `X-Frame-Options: SAMEORIGIN` on the host; plugin iframes served from `/plugins/` path with 1-hour cache.

- **OAuth2**: Familiar with PKCE flows — implementing for Spotify plugin

---

Phase 2: Architecture Discovery

6. Plugin Architecture

Decision: Iframe-based isolation with postMessage protocol

Approach	Security	DX Complexity	Performance	Chosen?
Iframes + postMessage	Strong (process isolation)	Medium	Good (parallel loading)	Yes
Web Components	Weak (shared JS context)	Low	Excellent	No
Server-side rendering	Strong (no client code)	High	Poor (latency per render)	No

How apps register tool schemas: Each plugin defines a `PluginDefinition` object with a `toolSchema` field that follows the OpenAI function calling format. The registry converts these to Zod schemas at startup and generates AI SDK tool objects.

```
PluginDefinition.toolSchema → paramToZod() → z.object() → ai.tool() → injected into
streamText()
```

Message passing protocol: Four-event postMessage lifecycle:

1. `plugin:ready` — iframe loaded, ready to receive invocations
2. `plugin:invoke` — host sends tool call arguments to the plugin
3. `plugin:state` — plugin reports intermediate state updates (e.g., chess board position changes)
4. `plugin:complete` — plugin signals task completion with result data
5. `plugin:error` — plugin reports an error

Runtime tool discovery: The plugin registry (`getPluginToolSet()`) is called at the start of each `streamText()` call. It iterates all registered plugins and returns a merged tool set. The AI model receives all available tool schemas in its system prompt and decides which to invoke based on user intent.

7. LLM and Function Calling

Provider: Multi-provider support (OpenAI, Anthropic, Google, + 20 others). The platform uses the Vercel AI SDK which normalizes function calling across providers. Any model with tool-use capability works.

Dynamic schema injection: Plugin tool schemas are merged into the tool set parameter of `streamText()` . This happens per-request, so the schema set can change between messages if plugins are enabled/disabled mid-session.

Context window management: Three mitigation strategies:

1. Each plugin tool schema adds only 60–120 tokens — lightweight by design
2. Automatic message compaction reduces conversation history when approaching context limits (configurable threshold: 40–90% of context window)

3. Teachers can disable [illegible]sed plugins to reduce per-request token overhead

Streaming with tool resu[illegible] [illegible]he AI SDK handles streaming natively. When the model emits a tool call, the stream pauses, the tool e[illegible]es (returning the plugin metadata), and the `PluginIframeUI` component renders the iframe. The c[illegible]sation continues when the model receives the tool result.

8. Real-Time Commu[illegible]tion

Decision: No WebSocket[illegible]ded

Layer	[illegible]	Protocol	Reason
Chat streaming	S[illegible] S[illegible]	[illegible] Sent Events (via AI S[illegible]	One-directional stream from LLM to client
App ↔ Platform	p[illegible]	[illegible]ssage	Browser-native, zero infrastructure, synchronous
State persistence	L[illegible]	[illegible]torage (IndexedDB)	No server-side session state needed

Bidirectional state upda[illegible] [illegible]he `plugin:state` message type lets apps push state changes to the host. The host stores these in [illegible] state (`useState` in `PluginFrame`) and can pass them back to the AI in subsequent turns.

Reconnection: Not applicable [illegible] reconnection. If an iframe fails [illegible] `plugin:ready` state.

9. State Management

Chat state: Zustand + Jotai sto[illegible] messages, and settings are sep[illegible]

App state: Owned entirely by t[illegible] this is a deliberate design choic[illegible] what flows through `plugin:state` [illegible] `plugin:complete`

Session state: Tool invocation [illegible] `MessageToolCallPart` objec[illegible] tool result is in the conversatio[illegible]

Page refresh: Chat history pers[illegible] stateless from the platform's pe[illegible] state" for the AI to reference.

10. Authentication Archite[illegible]

Platform auth: The base Chatb[illegible] their own API keys (BYOK mode[illegible]

Per-app auth: Three tiers as re[illegible]

App Type	[illegible]

Internal	None	Bundled with platfor	Chess
External (Public)	API key or none	Key passed via env v t build time	Weather en-Meteo, f
External (Authenticated)	OAuth2 PKCE	Plugin initiates OAut y via `allow-popups`	Spotify

Token storage: OAuth t s for authenticated plugins are manag ithin the plugin iframe n storage context (localStorage sc to the iframe origin). The platform d ot handle or store thi rty OAuth tokens — this maintains olation boundary.

OAuth redirect handlin e iframe has `allow-popups` in its s ox, so OAuth flows op a new window. The redirect URL points back to the plugin's origin. The p handles the token ex e internally and reports success `plugin:complete`

11. Database and Persist

No traditional database requ

Data	
Conversation history	In
User settings	In (d
Plugin registrations	In-
Tool invocation history	En

Schema for conversations: E tool name, arguments, and res invocation history.

Read/write patterns: Write-h session load (full history retrie

---

Phase 3: Post-Stack

12. Security and Sandbo

Iframe sandbox attributes:

```
<iframe sandbox="allow-scripts allow-same-origin all     opups allow-forms" />
```

Attribute	
`allow-scripts`	Plug

`allow-same-origin`	Plugin can access its own localStorage	Scoped to plugin origin, cannot access host
`allow-popups`	OAuth flows open in new window	Popups blocked by default; only used for auth
`allow-forms`	Form submission within plugin	Scoped to plugin iframe
~~`allow-top-navigation`~~	**Not included**	Prevents plugin from redirecting the host page

CSP headers (configured in `vercel.json`):

- `X-Frame-Options: SAMEORIGIN` — prevents the host from being embedded elsewhere
-

- End-to-end invoc lifecycle: user message → AI tool call → p render → user interaction → completion → fol p
- Multi-app switch tart chess, switch to weather, return to che
- Context retentior about a previous plugin interaction

Load testing: Not pric d for the sprint. The architecture is client- so load scales horizontally by user count (each user heir own AI connection and plugin instance

Testing commands:

```
pnpm test              # Run all tests
pnpm test:watch        # Watch mode
pnpm test:covera       # Coverage report
pnpm test:integr       # Integration tests (300s timeou
```

15. Developer Exp ce

Building a third-party requires:

1. Define a `Plugin nition` with tool schema (JSON-like stru
2. Create an HTML at implements the postMessage protocol
3. Listen for `plugi voke`, post `plugin:ready` on load, post `gin:complete` when done

**Minimal plugin exam omplete working plugin in ~30 lines of HT

```
<script>
  window.addEven ener('message', (e) => {
    if (e.data.t === 'plugin:invoke') {
      // Do work e.data.args
      window.par ostMessage({
        type: 'p :complete',
        pluginId ata.pluginId,
        toolCall .data.toolCallId,
        result: cess: true }
      }, '*');
    }
  });
  window.parent. essage({ type: 'plugin:ready', plu d: 'my-plugin' }, '*');
</script>
```

Local development: Run `pnpm dev:web` for hot reload. Plugin HTM in `public/plugins/` are served directly by Vite. Char

Debugging: Browser DevTo
console. Plugin errors flow b `plugin:error`

16. Deployment and Op

Platform deployment: Stat m run build:web
`release/app/dist/renderer/` `vercel.j`

Third-party app hosting: B `public/pl /`
platform. External third-part `iframeUrl`

`PluginDefinit` [redacted] rts absolute URLs.

CI/CD: GitLab CI [redacted] ted builds. Pipeline s[redacted]

1. Lint (`pnpm` [redacted]
2. Type check [redacted] `eck`)
3. Test (`pnpm` [redacted]
4. Build (`pnpm` [redacted] `eb`)
5. Deploy to Ve[redacted]

Monitoring: Sent[redacted] on for error tracking [redacted] `allowReport` [redacted] `dTracking` boundary at the React [redacted] onent tree root captu[redacted] `plugin:error` postM[redacted]

App updates without [redacted] is an atomic operation [redacted] are re-rendered. Plugin [redacted] `version` [redacted] `PluginDefinition` if needed.

Technical Stac[redacted]

Layer	[redacted]
Base Platform	[redacted]
Frontend	[redacted]
State	[redacted]
Build	[redacted]
AI Integration	[redacted]
App Sandboxing	[redacted]
Auth (Platform)	[redacted]
Auth (Plugins)	[redacted]
Testing	[redacted]
Deployment	[redacted]

Build Priority O[redacted]

Priority	[redacted]

1	Basic chat (streaming, history, context)	Done (from Chatbox fork)
2	Plugin registration + tool schema contract	Done
3	Tool invocation (AI discovers and calls plugin tools)	Done
4	UI embedding (iframe renders inline in chat)	Done
5	Completion signaling (plugin → chatbot lifecycle)	Done
6	Context retention (chatbot remembers plugin results)	Done
7	Multiple apps (Chess + Weather + Spotify)	Done
8	Auth flows (Spotify OAuth2 PKCE)	Done
9	Error handling (timeouts, crashes, invalid calls)	In progress
10	Developer docs (API documentation for third-party devs)	In progress